

0x000

EIXC4LRANZXXOIDUNBSW4LBAPFXXKIDDNRSWC3RAORUGKIDPOV2HG2LEMUQG6ZRAORUGKIDDOVY
CAYLOMQQGI2LTNAWCAYTVOQQGS3TTNFSGKIDZN52SAYLSMUQGM5LMNQQG6ZRAM5ZGKZLEEBQW4
ZBAO5UWG23FMRXGK43TFYRCA===

TGV0IHNVbWVvbmUgZWxzZSBwcmFpc2UgeW91LCBhbmQgbm90IHlvdXIgb3dulG1vdXRoOyBhbiBv
dXRzaWRlciwgYW5kIG5vdCB5b3VyIG93biBsaXBzLgo=

446f206e6f74206578616c7420796f757273656c6620696e20746865206b696e67e28099732070726573656e6365
2c20616e6420646f206e6f7420636c61696d206120706c61636520616d6f6e6720686973206772656174206d656e
3b2069742069732062657474657220666f722068696d20746f2073617920746f20796f752c20e2809c436f6d65207
57020686572652ce2809d207468616e20666f722068696d20746f2068756d696c6961746520796f75206265666f7
26520686973206e6f626c65732e

automobiles, alcohol, blood, sweat and creative reversing of an obfuscated Car-Modding tool

by atlas @atlas
atlas@r4780y.com

a little about me

- who is this atlas dork?
- more importantly, why do i care?
 - hacker of things
 - maker of hacks
 - biker
 - father
 - Jesus-dude





owner of things
GRIMM

Email atlas@r4780y.com

Email 0a71a5@gmail.com

Facebook <https://www.facebook.com/john.seldom.16/>

Twitter [atlas](#)

Website <http://atlas.r4780y.com>

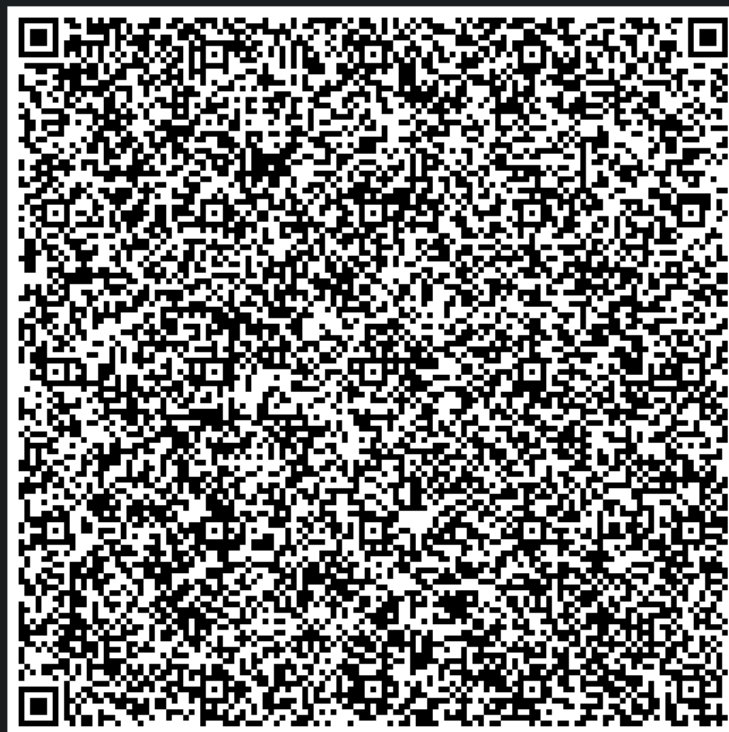
4099 Cedar Commercial Dr
CEDAR SPRINGS, MI 40319
UNITED STATES

Profession exploitation

Department Cyber Physical, R&D

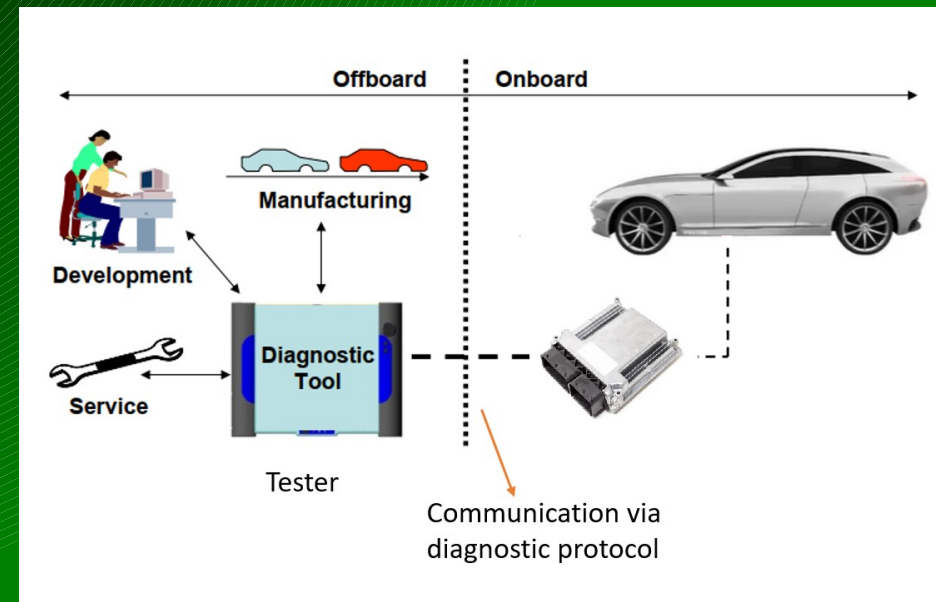
Address Book Personal Contacts

Note <https://github.com/atlas0fd00m>
<https://github.com/atlas0fd00m/rfcat>
<https://github.com/atlas0fd00m/vivisect>
<https://github.com/atlas0fd00m/VivisectION>



RCE on automotive ECUs?

- tool chains to update firmware, change performance parms
 - Windows
 - Vendor Software
 - Universal Diagnostics interface
 - hardware that talks to the car
 - OBD2
 - in-vehicle network (CAN)
 - ECU



authentication?

- **set a diagnostics level (SVC 0x10)**
- **Security Access (0x27) or Authentication (0x29)**
 - **seed/key exchange (SecAccess)**

```
vcan0 7E1 [3] [SF] ln: 2 data: 10 02 - '..' - [SRQ] DiagnosticSessionControl
vcan0 7E9 [3] [SF] ln: 2 data: 50 02 - 'P.' - [PSR] DiagnosticSessionControl
vcan0 7E1 [3] [SF] ln: 2 data: 27 01 - '.' - [SRQ] SecurityAccess
vcan0 7E9 [7] [SF] ln: 6 data: 67 01 5E 88 F6 8D - 'g.^ ...' - [PSR] SecurityAccess
vcan0 7E1 [7] [SF] ln: 6 data: 27 02 4C BC A0 F5 - 'L ...' - [SRQ] SecurityAccess
vcan0 7E9 [3] [SF] ln: 2 data: 67 02 - 'g.' - [PSR] SecurityAccess
vcan0 7E1 [8] [FF] ln: 8 data: 23 24 20 00 00 00 - '$ ...' - [SRQ] ReadMemoryByAddress
vcan0 7E9 [3] [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x00 = 0 ms
vcan0 7E1 [3] [CF] sn: 1 data: 00 04 - '..' -
vcan0 7E9 [6] [SF] ln: 5 data: 63 00 00 00 00 - 'c....' - [PSR] ReadMemoryByAddress
vcan0 7E1 [8] [FF] ln: 12 data: 3D 24 20 00 00 00 - '$ ...' - [SRQ] WriteMemoryByAddress
vcan0 7E9 [3] [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x00 = 0 ms
vcan0 7E1 [7] [CF] sn: 1 data: 00 04 01 02 03 04 - '.....' -
vcan0 7E9 [8] [FF] ln: 8 data: 7D 24 20 00 00 00 - '$ ...' - [PSR] WriteMemoryByAddress
vcan0 7E1 [8] [FF] ln: 8 data: 23 24 20 00 00 00 - '$ ...' - [SRQ] ReadMemoryByAddress
vcan0 7E9 [3] [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x00 = 0 ms
vcan0 7E1 [3] [CF] sn: 1 data: 00 04 - '..' -
vcan0 7E9 [6] [SF] ln: 5 data: 63 04 03 02 01 - 'c....' - [PSR] ReadMemoryByAddress
```

auth: the old way: UDS service 0x27: Security Access

- simple seed / key exchange
- tool requests seed, ECU returns random value
- tool processes seed through MATH
- tool sends munged response back to ECU
- ECU checks to make sure the MATH is correct

auth: newer way: UDS service 0x29: Authentication

- allows for more powerful auth, including public-key crypto, etc.
- implementation details matter
- once you know how, you may be able to mirror the tool's behavior

the goal: Discover UDS Authentication Methods

the target: Binary File - obfuscated PE binary, no dependencies

```
:0x00401000 Segment: (3035136 bytes)
:0x00401000
:0x00401000 FUNC: int cdecl forscan.__entry( ) [0 XREFS]
:0x00401000
:0x00401000 Stack Variables: (offset from initial top of stack)
:0x00401000
:0x00401000 6801908000 push loc_00809001
:0x00401005 e801000000 call sub_0040100b ;sub_0040100b()
:0x0040100a c3 ret
```

```
:0x0040100b
:0x0040100b FUNC: int cdecl sub_0040100b( ) [2 XREFS]
:0x0040100b
:0x0040100b Stack Variables: (offset from initial top of stack)
:0x0040100b
:0x0040100b c3 ret
```

```
.data:0x00809000 Segment: .data (507904 bytes)
.data:0x00809000 90
.data:0x00809001 loc_00809001: [1 XREFS]
.data:0x00809001 60
.data:0x00809002 e8 .data:0x00809000 Segment: .data (507904 bytes)
.data:0x00809003 03 .data:0x00809000 90
.data:0x00809004 00 .data:0x00809001 loc_00809001: [1 XREFS]
.data:0x00809005 00 .data:0x00809001 60 pushad
.data:0x00809002 e803000000 call sub_0080900a
.data:0x00809007 e9eb045d45 jmp 0x45dd94f7
.data:0x0080900c 55 push ebp
.data:0x0080900d c3 ret
.data:0x0080900e e8
.data:0x0080900f 01
```

```
.data:0x00809000 Segment: .data (507904 bytes)
.data:0x00809000 90
.data:0x00809001 loc_00809001: [1 XREFS]
.data:0x00809001 60 pushad
.data:0x00809002 e803000000 call sub_0080900a
.data:0x00809007 e9
.data:0x00809008 eb
.data:0x00809009 04
.data:0x0080900a
.data:0x0080900a FUNC: int cdecl sub_0080900a( ) [1 XREFS]
.data:0x0080900a
.data:0x0080900a Stack Variables: (offset from initial top of stack)
.data:0x0080900a
.data:0x0080900a 5d pop ebp
.data:0x0080900b 45 inc ebp
.data:0x0080900c 55 push ebp
.data:0x0080900d c3 ret
```

the makings of a very long day

interesting ways to approach:

- emulation
- "snatch-n-grab"

emulation (not for this talk)

- QEMU?
- kernel debugging?
- VivisectIION :)
- but not this time!

"snatch-n-grab" - running it, and "kidnapping" the process RAM

- starting it up
- connecting
- attaching with a debugger
 - could potentially defend against this someday.
 - but then, kernel debugging (probably my next talk: QEMU on Linux to WinDBG Kernel debugging)
- slurping up the data

"snatch-n-grab" - running it, and "kidnapping" the process RAM

- starting it up
- connecting
- attaching with a debugger
 - could potentially defend against this someday.
 - but then, kernel debugging (probably my next talk: QEMU on Linux to WinDBG Kernel debugging)
- slurping up the data

- Vivisect's VDB
- Saving a VSnapshot
 - Converting the VSnapshot into a Vivisect Workspace
 - magic...



```
vdb > attach 5212
Attaching to 5212
vdb > Attached to : 5212
Loading Binary: C:\Program Files\
Loading Binary: C:\Windows\System32\ntdll.dll
New Thread: 5220
New Thread: 5232
New Thread: 5236
New Thread: 5616
New Thread: 5620
New Thread: 5624
Loading Binary: C:\Windows\System32\kernel32.dll
Loading Binary: C:\Windows\System32\KernelBase.dll
Loading Binary: C:\Windows\System32\ws2_32.dll
Loading Binary: C:\Windows\System32\msvcrt.dll
Loading Binary: C:\Windows\System32\rpcrt4.dll
Loading Binary: C:\Windows\System32\ntsi.dll
Loading Binary: C:\Windows\System32\shlwapi.dll
Loading Binary: C:\Windows\System32\gdi32.dll
Loading Binary: C:\Windows\System32\user32.dll
Loading Binary: C:\Windows\System32\lpk.dll
Loading Binary: C:\Windows\System32\usp10.dll
Loading Binary: C:\Windows\System32\wininet.dll
Loading Binary: C:\Windows\System32\api-ms-win-downlevel-user32-l1-1-0.dll
Loading Binary: C:\Windows\System32\api-ms-win-downlevel-shlwapi-l1-1-0.dll
Loading Binary: C:\Windows\System32\api-ms-win-downlevel-version-l1-1-0.dll
Loading Binary: C:\Windows\System32\version.dll
Loading Binary: C:\Windows\System32\api-ms-win-downlevel-normaliz-l1-1-0.dll
Loading Binary: C:\Windows\System32\normaliz.dll
Loading Binary: C:\Windows\System32\iertutil.dll
Loading Binary: C:\Windows\System32\api-ms-win-downlevel-advapi32-l1-1-0.dll
Loading Binary: C:\Windows\System32\advapi32.dll
Loading Binary: C:\Windows\System32\sechost.dll
Loading Binary: C:\Windows\System32\userenv.dll
Loading Binary: C:\Windows\System32\profapi.dll
Loading Binary: C:\Windows\System32\shell32.dll
Loading Binary: C:\Windows\System32\winmm.dll
Loading Binary: C:\Windows\System32\oleaut32.dll
Loading Binary: C:\Windows\System32\ole32.dll
Loading Binary: C:\Windows\System32\imm32.dll
Loading Binary: C:\Windows\System32\msctf.dll
Loading Binary: C:\Windows\System32\wsock32.dll
Loading Binary: C:\Windows\System32\uxtheme.dll
Loading Binary: C:\Windows\System32\wintrust.dll
Loading Binary: C:\Windows\System32\crypt32.dll
Loading Binary: C:\Windows\System32\msasn1.dll
Loading Binary: C:\Windows\System32\dwapi.dll
Loading Binary: C:\Windows\System32\cryptbase.dll
New Thread: 5888
Thread: 5888 NOTIFY_BREAK
```

Loading Binary: C:\\Windows\\System32\\imm32.dll
Loading Binary: C:\\Windows\\System32\\msctf.dll
Loading Binary: C:\\Windows\\System32\\wsock32.dll
Loading Binary: C:\\Windows\\System32\\uxtheme.dll
Loading Binary: C:\\Windows\\System32\\wintrust.dll
Loading Binary: C:\\Windows\\System32\\crypt32.dll
Loading Binary: C:\\Windows\\System32\\msasn1.dll
Loading Binary: C:\\Windows\\System32\\dwmapi.dll
Loading Binary: C:\\Windows\\System32\\cryptbase.dll

New Thread: 5888
Thread: 5888 NOTIFY_BREAK

Snapshot Target.vsnap

Taking Snapshot...

Comparing 0x10000 to 0x3f80000-0x3f7f000

Comparing 0x10000 to 0x40a0000-0x409f000

Comparing 0x10000 to 0x42c0000-0x42bf000

Comparing 0x10000 to 0x43c0000-0x43bf000

Comparing 0x10000 to 0x44c0000-0x44bf000

Comparing 0x10000 to 0x45c0000-0x45bc000

Comparing 0x20000 to 0x130000-0x122000

Comparing 0x20000 to 0x34b0000-0x34af000

Comparing 0x20000 to 0x3f80000-0x3f7f000

Comparing 0x20000 to 0x40a0000-0x409f000

Comparing 0x20000 to 0x42c0000-0x42bf000

Comparing 0x20000 to 0x43c0000-0x43bf000

Comparing 0x20000 to 0x44c0000-0x44bf000

Comparing 0x20000 to 0x45c0000-0x45bc000

Comparing 0x121000 to 0x130000-0x122000

Comparing 0x121000 to 0x34b0000-0x34af000

Comparing 0x121000 to 0x3f80000-0x3f7f000

Comparing 0x121000 to 0x40a0000-0x409f000

Comparing 0x121000 to 0x42c0000-0x42bf000

Comparing 0x121000 to 0x43c0000-0x43bf000

Comparing 0x121000 to 0x44c0000-0x44bf000

Comparing 0x121000 to 0x45c0000-0x45bc000

Comparing 0x122000 to 0x130000-0x122000

Comparing 0x122000 to 0x34b0000-0x34af000

Comparing 0x122000 to 0x3f80000-0x3f7f000

Comparing 0x122000 to 0x40a0000-0x409f000

Comparing 0x122000 to 0x42c0000-0x42bf000

Comparing 0x122000 to 0x43c0000-0x43bf000

Comparing 0x122000 to 0x44c0000-0x44bf000

Comparing 0x122000 to 0x45c0000-0x45bc000

New Inread: 5888
Thread: 5888 NOTIFY_BREAK
Snapshot Target.vsnap
Taking Snapshot...
Comparing 0x10000 to 0x130000-0x

Comparing 0x7ffd9000 to 0x3f80000-0x3f7f000
Comparing 0x7ffd9000 to 0x40a0000-0x409f000
Comparing 0x7ffd9000 to 0x42c0000-0x42bf000
Comparing 0x7ffd9000 to 0x43c0000-0x43bf000
Comparing 0x7ffd9000 to 0x44c0000-0x44bf000
Comparing 0x7ffd9000 to 0x45c0000-0x45bc000
Comparing 0x7ffda000 to 0x130000-0x122000
Comparing 0x7ffda000 to 0x34b0000-0x34af000
Comparing 0x7ffda000 to 0x3f80000-0x3f7f000
Comparing 0x7ffda000 to 0x40a0000-0x409f000
Comparing 0x7ffda000 to 0x42c0000-0x42bf000
Comparing 0x7ffda000 to 0x43c0000-0x43bf000
Comparing 0x7ffda000 to 0x44c0000-0x44bf000
Comparing 0x7ffda000 to 0x45c0000-0x45bc000
Comparing 0x7ffdb000 to 0x130000-0x122000
Comparing 0x7ffdb000 to 0x34b0000-0x34af000
Comparing 0x7ffdb000 to 0x3f80000-0x3f7f000
Comparing 0x7ffdb000 to 0x40a0000-0x409f000
Comparing 0x7ffdb000 to 0x42c0000-0x42bf000
Comparing 0x7ffdb000 to 0x43c0000-0x43bf000
Comparing 0x7ffdb000 to 0x44c0000-0x44bf000
Comparing 0x7ffdb000 to 0x45c0000-0x45bc000
Comparing 0x7ffdd000 to 0x130000-0x122000
Comparing 0x7ffdd000 to 0x34b0000-0x34af000
Comparing 0x7ffdd000 to 0x3f80000-0x3f7f000
Comparing 0x7ffdd000 to 0x40a0000-0x409f000
Comparing 0x7ffdd000 to 0x42c0000-0x42bf000
Comparing 0x7ffdd000 to 0x43c0000-0x43bf000
Comparing 0x7ffdd000 to 0x44c0000-0x44bf000
Comparing 0x7ffde000 to 0x130000-0x122000
Comparing 0x7ffde000 to 0x34b0000-0x34af000
Comparing 0x7ffde000 to 0x3f80000-0x3f7f000
Comparing 0x7ffde000 to 0x40a0000-0x409f000
Comparing 0x7ffde000 to 0x42c0000-0x42bf000
Comparing 0x7ffde000 to 0x43c0000-0x43bf000
Comparing 0x7ffde000 to 0x44c0000-0x44bf000
Comparing 0x7ffde000 to 0x45c0000-0x45bc000
Comparing 0x7ffdf000 to 0x130000-0x122000
Comparing 0x7ffdf000 to 0x34b0000-0x34af000
Comparing 0x7ffdf000 to 0x3f80000-0x3f7f000
Comparing 0x7ffdf000 to 0x40a0000-0x409f000
Comparing 0x7ffdf000 to 0x42c0000-0x42bf000
Comparing 0x7ffdf000 to 0x43c0000-0x43bf000
Comparing 0x7ffdf000 to 0x44c0000-0x44bf000
Comparing 0x7ffdf000 to 0x45c0000-0x45bc000
Comparing 0x7fffe000 to 0x130000-0x122000
Comparing 0x7fffe000 to 0x34b0000-0x34af000
Comparing 0x7fffe000 to 0x3f80000-0x3f7f000
Comparing 0x7fffe000 to 0x40a0000-0x409f000
Comparing 0x7fffe000 to 0x42c0000-0x42bf000
Comparing 0x7fffe000 to 0x43c0000-0x43bf000
Comparing 0x7fffe000 to 0x44c0000-0x44bf000
Comparing 0x7fffe000 to 0x45c0000-0x45bc000
Saving To File
Done
vdb >

importing VSnapshot into a Vivisect Workspace

manual process
soon in VivisectION
eventually Vivisect

2-stages
Hard Fought
Lessons

```
# migrate all memory maps
for mmva, mmsz, mmperm, mmname in snap.getMemoryMaps():
    if vw.getMemoryMap(mmva):
        # skipping existing map
        continue

    logger.warning("=== Loading Raw Memory Map: %x:%x %x: %r", mmva, mmsz, mmperm, mmname)
    bytez = snap.readMemory(mmva, mmsz)

    # temp PE
    pe = None
    try:
        if snap.readMemory(mmva, 2) != b'MZ':
            pe = PE.peFromMemoryObject(snap, mmva)
            if pe.IMAGE_DOS_HEADER.e_magic != 0x00005a4d:
                logger.warning("Wrong DOS HDR Magic.  Skipping")
                pe = None

    except Exception as e:
        pass

    # Add the memory maps in
    mmname = normalizeFname(mmname, pe)
    vw.addMemoryMap(mmva, mmperm, mmname, bytez)
```

importing vsnap into VivWorkspace (part 2)

```
# now go back and parse the symbols/relocs/etc
for mmva, mmsz, mmperm, mmname in snap.getMemoryMaps():
    try:
        if snap.readMemory(mmva, 2) != b'MZ':
            continue

        pe = PE.peFromMemoryObject(snap, mmva)
        if pe.IMAGE_DOS_HEADER.e_magic != 0x00005a4d:
            logger.warning("Wrong DOS HDR Magic.  Skipping")
            continue

        logger.warning("=== Trying Memory Map (as PE) %x:%x %x: %r", mmva, mmsz, mmperm, mmname)
        applyFromParser(vw, mmva, pe, mmname)

    except Exception as e:
        print(e)
        import traceback
        traceback.print_exc()
```

```

def applyFromParser(vw, baseaddr, pe, fname):
    # PE only
    # Setup some va sets used by windows analysis modules
    # these will already exist for Multi-file workspaces
    vaSetNames = vw.getVaSetNames()
    if not 'Library Loads' in vaSetNames:
        vw.addVaSet('Library Loads', (('Address', VASET_ADDRESS), ('Library', VASET_STRING)))
    if not 'pe:ordinals' in vaSetNames:
        vw.addVaSet('pe:ordinals', (('Address', VASET_ADDRESS), ('Ordinal', VASET_INTEGER)))
    if not 'DelayImports' in vaSetNames:
        vw.addVaSet('DelayImports', (('Address', VASET_ADDRESS), ('DelayImport', VASET_STRING)))

    byts = pe.getFileBytes()

    if not vw.getMeta("Architecture"):
        mach = pe.IMAGE_NT_HEADERS.FileHeader.Machine

        arch = vppe.arch_names.get(mach)
        if arch is None:
            raise v_exc.InvalidArchitecture("PE", hex(mach))

        vw.setMeta('Architecture', arch)
        vw.setMeta('Format', 'pe')
        vw.parsedbin = pe

        platform = 'windows'

        # Drivers are platform "winkern" so impapi etc works
        subsys = pe.IMAGE_NT_HEADERS.OptionalHeader.Subsystem
        if subsys == PE.IMAGE_SUBSYSTEM_NATIVE:
            platform = 'winkern'

        vw.setMeta('Platform', platform)

        vw.setMeta('DefaultCall', archcalls.get(arch, 'unknown'))
        logger.info("PE loader: Arch: %r\tFormat: pe\tPlatform: %r\tFilename: %r\tBaseAddr: 0x%x", \
                    arch, platform, fname, baseaddr)

    if fname not in vw.GetFiles():
        # Set ourselves up for extended windows binary analysis

        imagebase = pe.IMAGE_NT_HEADERS.OptionalHeader.ImageBase
        if baseaddr is None:
            baseaddr = imagebase

        entryrva = pe.IMAGE_NT_HEADERS.OptionalHeader.AddressOfEntryPoint
        entry = entryrva + baseaddr

        codebase = pe.IMAGE_NT_HEADERS.OptionalHeader.BaseOfCode
        codesize = pe.IMAGE_NT_HEADERS.OptionalHeader.SizeOfCode
        codervamax = codebase+codesize

        logger.info("PE Imagebase: 0x%x\tentry: 0x%x\tcodebase: 0x%x\tcodesize: 0x%x", \
                    imagebase, entry, codebase, codesize)
        fhash = v_parsers.md5Bytes(byts)
        sha256 = v_parsers.sha256Bytes(byts)

        fvvname = fname

        # This will help linkers with files that are re-named
        dllname = pe.getDllName()
        if dllname:
            fvvname = dllname

        if fvvname is None:
            fvvname = fhash

        logger.info("PE dllname: %r\tfvvname: %r\tmd5: %r\tsha256: %r", \
                    dllname, fvvname, fhash, sha256)

        # create the file and store md5 and sha256 hashes
        fname = vw.addFile(fvvname.lower(), baseaddr, fhash)

```

modified from vivisect/parser/pe.py

```

        vw.setFileMeta(fname, 'sha256', sha256)
        vw.setFileMeta(fname, 'FileBytes', v_parsers.compressBytes(byts))

    symhash = e_symcache.symCacheHashFromPe(pe)
    vw.setFileMeta(fname, 'SymbolCacheHash', symhash)

    # Add file version info if VS_VERSIONINFO has it
    try:
        vs = pe.getVS_VERSIONINFO()
    except Exception as e:
        vs = None
        vw.vprint('Failed to load version info resource due to %s' % (repr(e)),)
    if vs is not None:
        vsver = vs.getVersionValue('FileVersion')
        if vsver is not None and len(vsver):
            # add check to split seeing samples with spaces and nothing else..
            parts = vsver.split()
            if len(parts):
                vsver = vsver.split()[0]
                vw.setFileMeta(fname, 'Version', vsver)

    # SizeOfHeaders Spoofable...
    curr_offset = pe.IMAGE_DOS_HEADER.e_lfanew + len(pe.IMAGE_NT_HEADERS)

    secsize = len(vstruct.getStructure("pe.IMAGE_SECTION_HEADER"))

    sec_offset = pe.IMAGE_DOS_HEADER.e_lfanew + 4 + len(pe.IMAGE_NT_HEADERS.FileHeader) + pe.IMAGE_NT_HEADERS.FileHeader.SizeOfOptionalHeader

    if sec_offset != curr_offset:
        header_size = sec_offset + pe.IMAGE_NT_HEADERS.FileHeader.NumberOfSections * secsize
    else:
        header_size = pe.IMAGE_DOS_HEADER.e_lfanew + len(pe.IMAGE_NT_HEADERS) + pe.IMAGE_NT_HEADERS.FileHeader.NumberOfSections * secsize

    # Add the first page mapped in from the PE header.
    header = pe.readAtOffset(0, header_size)

    if not header:
        raise v_exc.CorruptPeFile("truncated PE header")

    secalign = pe.IMAGE_NT_HEADERS.OptionalHeader.SectionAlignment
    filealign = pe.IMAGE_NT_HEADERS.OptionalHeader.FileAlignment
    subsys_majver = pe.IMAGE_NT_HEADERS.OptionalHeader.MajorSubsystemVersion
    subsys_minver = pe.IMAGE_NT_HEADERS.OptionalHeader.MinorSubsystemVersion

    if secalign == 0:
        raise v_exc.CorruptPeFile("section alignment is zero")

    secrem = len(header) % secalign
    if secrem != 0:
        header += b'\x00' * (secalign - secrem)

    #vw.addMemoryMap(baseaddr, e_const.MM_READ, fname, header)
    vw.addSegment(baseaddr, len(header), "PE_Header", fname)

    hstruct = vw.makeStructure(baseaddr, "pe.IMAGE_DOS_HEADER")
    magicaddr = hstruct.e_lfanew
    if vw.readMemory(baseaddr + magicaddr, 2) != b"PE":
        raise Exception("We only support PE exe's")

    if not vw.isLocation(baseaddr + magicaddr):
        padloc = vw.makePad(baseaddr + magicaddr, 4)

    ifhdr_va = baseaddr + magicaddr + 4
    ifstruct = vw.makeStructure(ifhdr_va, "pe.IMAGE_FILE_HEADER")
    ohstruct = vw.makeStructure(ifhdr_va + len(ifstruct), "pe.IMAGE_OPTIONAL_HEADER")
    nxcompat = ohstruct.DLLCharacteristics & PE.IMAGE_DLLCHARACTERISTICS_NX_COMPAT

    # get resource data directory
    ddir = pe.getDataDirectory(PE.IMAGE_DIRECTORY_ENTRY_RESOURCE)
    loadsrc = vw.config.viv.parsers.pe.loadresources
    carvepes = vw.config.viv.parsers.pe.carvepes

```

```

deaddirs = [PE.IMAGE_DIRECTORY_ENTRY_EXPORT,
            PE.IMAGE_DIRECTORY_ENTRY_IMPORT,
            PE.IMAGE_DIRECTORY_ENTRY_RESOURCE,
            PE.IMAGE_DIRECTORY_ENTRY_EXCEPTION,
            PE.IMAGE_DIRECTORY_ENTRY_SECURITY,
            PE.IMAGE_DIRECTORY_ENTRY_BASERELOC,
            PE.IMAGE_DIRECTORY_ENTRY_DEBUG,
            PE.IMAGE_DIRECTORY_ENTRY_COPYRIGHT,
            PE.IMAGE_DIRECTORY_ENTRY_ARCHITECTURE,
            PE.IMAGE_DIRECTORY_ENTRY_GLOBALPTR,
            PE.IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG,
            PE.IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT,
            PE.IMAGE_DIRECTORY_ENTRY_IAT,
            PE.IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT,
            PE.IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR]

deadvas = [ddir.VirtualAddress]
for datadir in deaddirs:
    d = pe.getDataDirectory(datadir)
    if d.VirtualAddress:
        deadvas.append(d.VirtualAddress)

for idx, sec in enumerate(pe.sections):
    mapflags = 0
    offset = sec.vsGetMeta("Offset", None)
    if offset:
        addr = baseaddr + offset
        vw.makeStructure(addr, 'pe.IMAGE_SECTION_HEADER')
        chars = sec.Characteristics
        if chars & PE.IMAGE_SCN_MEM_READ:
            mapflags |= e_const.MM_READ

        isrsrc = (sec.VirtualAddress == ddir.VirtualAddress)
        if isrsrc and not loadrsrc:
            continue

        # If it's for an older system, just about anything
        # is executable...
        # However, there is the DLLCHARACTERISTICS NXCOMPAT flag to take into account,
        # which works with the OS to prevent certain pages of memory from achieving
        # execution unless they're marked with the execute bit
        # so we can't just blindly mark these as executable quite yet.
        if not nxcompat:
            if not vw.config.viv.parsers.pe.nx and subsys_majver < 6 and not isrsrc:
                mapflags |= e_const.MM_EXEC

    if chars & PE.IMAGE_SCN_MEM_READ:
        mapflags |= e_const.MM_READ
    if chars & PE.IMAGE_SCN_MEM_WRITE:
        mapflags |= e_const.MM_WRITE
    if chars & PE.IMAGE_SCN_MEM_EXECUTE:
        mapflags |= e_const.MM_EXEC
    if chars & PE.IMAGE_SCN_CNT_CODE:
        mapflags |= e_const.MM_EXEC
    if chars & PE.IMAGE_SCN_CNT_UNINITIALIZED_DATA:
        mapflags |= e_const.MM_UNINIT

    secrva = sec.VirtualAddress
    secvsize = sec.VirtualSize
    secfsize = sec.SizeOfRawData
    secbase = secrva + baseaddr
    secname = sec.Name.strip("\x00")
    secrvamax = secrva + secvsize

    # If the section is part of BaseOfCode->SizeOfCode
    # force execute perms...
    if secrva >= codebase and secrva < codervamax:
        mapflags |= e_const.MM_EXEC

    # If the entry point is in this section, force execute
    # permissions.
    if secrva <= entryrva and entryrva < secrvamax:

```

```

        mapflags |= e_const.MM_EXEC

    if not nxcompat:
        if not vw.config.viv.parsers.pe.nx and subsys_majver < 6 and mapflags & e_const.MM_READ:
            mapflags |= e_const.MM_EXEC

    if sec.VirtualSize == 0 or sec.SizeOfRawData == 0:
        if idx+1 >= len(pe.sections):
            continue
        # fill the gap with null bytes..
        nsec = pe.sections[idx+1]
        nbase = nsec.VirtualAddress + baseaddr

        plen = nbase - secbase
        readsize = sec.SizeOfRawData if sec.SizeOfRawData < sec.VirtualSize else sec.VirtualSize
        secoff = pe.rvaToOffset(secrva)
        secbytes = pe.readAtOffset(secoff, readsize)
        secbytes += b'\x00' * plen
        # PROBABLY CAN CLEAN OUT MORE OF THIS CODE
        #vw.addMemoryMap(secbase, mapflags, fname, secbytes)
        vw.addSegment(secbase, len(secbytes), secname, fname)

    # Mark dead data on resource and import data directories
    if sec.VirtualAddress in deadvas:
        vw.markDeadData(secbase, secbase+len(secbytes))

    #FIXME create a mask for this
    if not (chars & PE.IMAGE_SCN_CNT_CODE) and not (chars & PE.IMAGE_SCN_MEM_EXECUTE) and not (chars & PE.IMAGE_SCN_MEM_WRITE):
        vw.markDeadData(secbase, secbase+len(secbytes))
    continue

    # if SizeOfRawData is greater than VirtualSize we'll end up using VS in our read..
    if sec.SizeOfRawData < sec.VirtualSize:
        if sec.SizeOfRawData > pe.filesize:
            continue

    #plen = sec.VirtualSize - sec.SizeOfRawData

    try:
        # According to http://code.google.com/p/corkami/wiki/PE#section_table if SizeOfRawData is larger than VirtualSize, VS is used..
        readsize = sec.SizeOfRawData if sec.SizeOfRawData < sec.VirtualSize else sec.VirtualSize

        secoff = pe.rvaToOffset(secrva)
        secbytes = pe.readAtOffset(secoff, readsize, shortok=True)
        slen = len(secbytes)
        secbytes += b'\x00' * (sec.VirtualSize - slen)

        #slen = vw.addMemoryMap(secbase, mapflags, fname, secbytes, align=filealign)
        slen = len(secbytes)
        vw.addSegment(secbase, slen, secname, fname)

    # Mark dead data on resource and import data directories
    if sec.VirtualAddress in deadvas:
        vw.markDeadData(secbase, secbase+len(secbytes))

    #FIXME create a mask for this
    if not (chars & PE.IMAGE_SCN_CNT_CODE) and not (chars & PE.IMAGE_SCN_MEM_EXECUTE) and not (chars & PE.IMAGE_SCN_MEM_WRITE):
        vw.markDeadData(secbase, secbase+len(secbytes))

    except Exception as e:
        logger.warning("Error Loading Section (%s size:%d rva:%.8x offset: %d): %s", secname, secfsize, secrva, secoff, e)

    vw.addExport(entry, EXP_FUNCTION, '__entry', fname)
    vw.addEntryPoint(entry)

    # store the actual reloc section virtual address
    reloc_va = pe.getDataDirectory(PE.IMAGE_DIRECTORY_ENTRY_BASERELOC).VirtualAddress
    if reloc_va:
        reloc_va += baseaddr

    vw.setFileMeta(fname, "reloc_va", reloc_va)

    for rva, rtype in pe.getRelocations():

```

end of eye-chart

- reach out if you want this
- should be in VivisectIOn shortly
- currently in
vivisection/shit.py

```
# map PE reloc to VIV reloc ( or dont... )
vtypedata = vppe.reloc.get(rtype)
if vtypedata is None:
    logger.info('Skipping PE Relocation type: %d at %d (no handler)', rtype, rva)
    continue

vtype, vsize = vtypedata

try:
    vtfmt = e_bits.getFormat(vsize)    # for PowerPC, will need to get Endianness
    mapoffset = vw.readMemoryFormat(rva + baseaddr, vtfmt)[0] - imagebase
except Exception as e:
    # the target address of the relocation is not accessible.
    # for example, it's not mapped, or split across sections, etc.
    # technically, the PE is corrupt.
    # by continuing on here, we are a bit more robust (but maybe incorrect)
    # than the Windows loader.
    #
    # discussed in:
    # https://github.com/vivisect/vivisect/issues/346
    logger.warning('Skipping invalid PE relocation: %d (%r)', rva, e)
    continue
else:
    logger.info('PE relocation: 0x%x -> %r+0x%x', baseaddr+rva, fname, mapoffset)
    vw.addRelocation(rva + baseaddr, vtype, mapoffset)

for rva, lname, lname in pe.getImports():
    if vw.probeMemory(rva + baseaddr, 4, e_const.MM_READ):
        vw.makeImport(rva + baseaddr, lname, lname)

for rva, lname, lname in pe.getDelayImports():
    if vw.probeMemory(rva + baseaddr, 4, e_const.MM_READ):
        vw.makeImport(rva + baseaddr, lname, lname)
    if lname != '*':
        lname = vw.normFileName(lname)
        vw.setVaSetRow('DelayImports', (rva + baseaddr, lname + '.' + lname))

exports = pe.getExports()
for rva, ord, name in exports:
    eva = rva + baseaddr

    # Functions exported by ordinal only have no name
    if not name:
        name = "Ordinal_" + str(ord)

    try:
        vw.setVaSetRow('pe:ordinals', (eva, ord))
        vw.addExport(eva, EXP_UNTYPED, name, fname)
        if vw.probeMemory(eva, 1, e_const.MM_EXEC):
            vw.addEntryPoint(eva)
    except Exception as e:
        vw.vprint('addExport Failed: %s.%s (0x%.8x): %s' % (fname, name, eva, e))

# Save off the ordinals...
vw.setFileMeta(fname, 'ordinals', exports)

fwd = pe.getForwarders()
for rva, name, forwardname in fwd:
    vw.makeName(rva+baseaddr, "forwarder_%s.%s" % (fname, name))
    vw.makeString(rva+baseaddr)

vw.setFileMeta(fname, 'forwarders', fwd)
```

```
def normalizeFname(fname, parser=None):
    # PE only currently.  \\ as path delim, case-insensitive

    fvivname = None

    if parser:
        dllname = parser.getDllName()
        if dllname:
            #logger.warning('dllname: %r (%r)', dllname, fname)
            fvivname = dllname

    if fvivname is None:
        fparts = fname.split("\\")
        fvivname = fparts[-1]
        #logger.warning('filename: %r (%r)', fvivname, fname)

    fvivname = fvivname.lower()
    if fvivname.endswith('.dll') or fvivname.endswith('.exe'):
        fvivname = fvivname[:-4]

    #logger.warning('fvivname: %r (%r)', fvivname, fname)
    return fvivname
```

several common ways of obfuscating

- unpack/decrypt “in-place”
- unpack/decrypt into a new memory map(s)
- process hollowing
- many variations...

def changed the original binary's memory maps

- Original:

0x401000	68 01 90 80 00 e8 01 00 00 00 c3 c3 68 29 02 1b	h.....h)..
0x401010	3b ee bd 3e 07 10 88 9f 4c 89 f3 10 db 87 14 f6	;...>....L.....
0x401020	e5 7b 37 17 ba 54 f4 8d 26 8e c6 ee ae 3f 2f 8d	.{7..T..&....?/.
0x401030	a9 77 df de 9c d7 1b 34 73 b3 8b af b4 66 33 22	.w.....4s....f3"
0x401040	1e 10 b1 8e 91 a6 37 2e ec 5c bd 18 01 d2 7a 05	7..\....z.
0x401050	d9 f8 ac c3 d5 3c 7d 80 a6 60 73 4e 2d 73 89 d7	<}...`sN-s..
0x401060	06 05 03 e4 44 27 3d 29 98 60 ef af d6 21 84 be	D'=).`.!!!!..
0x401070	0a cd 7e 84 2d 27 27 5c a4 2e 11 35 b4 77 cd 8c	..~.-''\...5.w..
0x401080	47 a0 ed 3a 32 5a 58 fa de 8c c0 52 b2 59 2d c6	G...:2ZX....R.Y-.
0x401090	bb 42 fa ef e6 15 80 9d 27 eb 25 3e 87 13 71 31	.B.....'.%>..q1
0x4010a0	a8 e2 f5 4d 73 71 48 d5 d2 2e 86 ae 2e 6c 72 b7	...MsqH.....lr.
0x4010b0	a9 19 41 53 31 6c f5 b9 8e bd 57 00 ec 9c ba 1e	..AS1l....W.....
0x4010c0	37 47 00 44 42 d6 07 89 13 f5 20 cd 47 38 eb 34	7G.DB.....G8.4
0x4010d0	79 0d c7 1d d4 46 ac 29 65 52 ac 32 aa 26 17 7b	y....F.)eR.2.&.{
0x4010e0	8d a3 82 af dd 3f 20 f5 38 ec 8c 3f b8 31 82 c0	? .8..?.1..
0x4010f0	20 8e f7 53 99 18 19 70 a5 18 b6 21 8b 71 36 48	..S...p...!.q6H

- Deobfuscated:

0x401000	c7 01 94 0b 79 00 e9 09 8b 27 00 cc cc cc cc cc	y....'.....
0x401010	56 8b f1 c7 06 94 0b 79 00 e8 f6 8a 27 00 f6 44	V.....y....'.D
0x401020	24 08 01 74 09 56 e8 86 8b 27 00 83 c4 04 8b c6	\$.t.V...'......
0x401030	5e c2 04 00 cc cc cc cc cc cc cc cc cc cc cc	^.....
0x401040	8b 44 24 0c 83 f8 04 8b 4c 24 08 8b 54 24 04 56	.D\$.....L\$..T\$.V
0x401050	57 72 14 8b 32 3b 31 75 12 83 e8 04 83 c1 04 83	Wr..2;1u.....
0x401060	c2 04 83 f8 04 73 ec 85 c0 74 5e 0f b6 32 0f b6	s...t^..2..
0x401070	39 2b f7 75 45 83 e8 01 83 c1 01 83 c2 01 85 c0	9+.uE.....
0x401080	74 47 0f b6 32 0f b6 39 2b f7 75 2e 83 e8 01 83	tG..2..9+.u....
0x401090	c1 01 83 c2 01 85 c0 74 30 0f b6 32 0f b6 39 2b	t0..2..9+
0x4010a0	f7 75 17 83 e8 01 83 c1 01 83 c2 01 85 c0 74 19	.u.....t.
0x4010b0	0f b6 32 0f b6 01 2b f0 74 0f 85 f6 b8 01 00 00	..2...+.t.....
0x4010c0	00 7f 08 5f 83 c8 ff 5e c3 33 c0 5f 5e c3 cc cc	..._...^..3._^...
0x4010d0	51 56 8b f1 6a 00 8d 4c 24 08 e8 fb 7d 27 00 8b	QV..j..L\$...}'..
0x4010e0	46 04 83 f8 ff 73 06 83 c0 01 89 46 04 8d 4c 24	F....s.....F..L\$
0x4010f0	04 e8 05 7e 27 00 5e 59 c3 cc cc cc cc cc cc cc	...~'.^Y.....

now what?

- search for UDS type things
- search for CAN related things
- search for 0x29 and 0x27 as immediate comparators
- oh wait. problem: where's the code?!?!?!?
 - i don't even know where strings are

Ion tools: CLI

viv: 0x75d2b4

History ▾

0x75d2b4

256

Viv ▾

Opts ▾

```
map:0x0075d2b3 00
map:0x0075d2b4 53 'S'
map:0x0075d2b5 45 'E'
map:0x0075d2b6 43 'C'
map:0x0075d2b7 55 'U'
map:0x0075d2b8 52 'R'
map:0x0075d2b9 49 'I'
map:0x0075d2ba 54 'T'
map:0x0075d2bb 59 'Y'
map:0x0075d2bc 5f '_'
map:0x0075d2bd 41 'A'
map:0x0075d2be 52 'R'
map:0x0075d2bf 52 'R'
map:0x0075d2c0 41 'A'
map:0x0075d2c1 59 'Y'
map:0x0075d2c2 00
map:0x0075d2c3 00
```

FuncGraph0: None (0x401000) viv: 0x75d2b4

0x00401000 is not in a function!
Welcome to Vivisect (Qt Edition)!
Random Tip: Hold <shift> to move using the mouse

Functions

Name ▾	Address	Size	Ref Count

Python Inter...

Segme...

Structu...

Strings

Workspace N...

Imports

Funci...

Exports

a different view

- some strings
- from loading file metadata
- but not very interesting for our purposes
- Viv hasn't marked anything interesting

File Edit View Share Tools Window Plugins

Ion tools: CLI

viv: 0x75d2b4

History ▾ 0x75d2b4 256

0x75d1f7	00 10 04 00 20 04 00 00 00 00 00 00 00 00 00 00	...X...X...p.X.
0x75d204	c0 df 58 00 d0 c0 58 00 10 b1 58 00 a0 54 58 00	..X...X...X..TX.
0x75d214	60 55 58 00 30 55 58 00 10 c1 58 00 f0 b8 58 00	`UX..0UX...X...X.
0x75d224	70 b3 58 00 f0 54 58 00 00 b5 58 00 10 55 58 00	p.X..TX...X..UX.
0x75d234	f0 55 58 00 00 01 53 00 50 ef 58 00 c0 f0 58 00	.UX...S.P.X...X.
0x75d244	10 e7 58 00 b0 e7 58 00 20 e9 58 00 90 e8 58 00	..X...X. .X...X.
0x75d254	e0 d2 58 00 00 d2 58 00 30 e4 58 00 00 d6 60 00	..X...X.0.X...`.
0x75d264	80 7b 59 00 f0 b9 58 00 c0 96 59 00 10 82 59 00	..{Y...X...Y...Y.
0x75d274	f0 9a 59 00 c0 9c 59 00 10 c5 58 00 a0 83 59 00	..Y...Y...X...Y.
0x75d284	50 86 59 00 c0 dc 58 00 d0 f5 58 00 40 00 00 00	P.Y...X...X.@...
0x75d294	44 50 46 5f 53 4f 4f 54 5f 50 43 54 5f 4f 4c 00	DPF_SOOT_PCT_OL.
0x75d2a4	44 50 46 5f 53 4f 4f 54 5f 50 43 54 5f 43 4c 00	DPF_SOOT_PCT_CL.
0x75d2b4	53 45 43 55 52 49 54 59 5f 41 52 52 41 59 00 00	SECURITY_ARRAY..
0x75d2c4	53 57 44 4c 5f 53 45 43 5f 4c 45 56 45 4c 00 00	SWDL_SEC_LEVEL..
0x75d2d4	53 57 44 4c 49 44 5f 51 56 5f 41 52 52 41 59 00	SWDLID_QV_ARRAY.
0x75d2e4	53 57 44 4c 49 44 5f 41 52 52 41 59 00 00 00 00	SWDLID_ARRAY....
0x75d2f4	53 57 44 4c 49 44 32 5f 51 56 00 00 53 57 44 4c	SWDLID2_QV..SWDL
0x75d304	49 44 32 5f 51 54 00 00 53 57 44 4c 49 44 32 00	TD? OT..SWDI TD?.

FuncGraph0: None (0x401000) viv: 0x75d2b4

0x00401000 is not in a function!
Welcome to Vivisect (Qt Edition)!
Random Tip: Hold <shift> to move using the mouse

Strings

Address ▾	String
0x75722c2f	'shlwapi.UrlEscapeW\x00'
0x75722c09	'shlwapi.UrlCreateFromPathW\x00'
0x75722be2	'shlwapi.UrlCombineW\x00'

Python Inter... Segme... Structu... Strings Workspace N... Import

original string discovery

```
$ strings -tx temp_2_5_WORK.viv |less
```

- MILLions of strings
- some aren't real strings
- radix numbers mean **nothing**
- find an interesting string
- from Vivisect command line:

```
search Extended License is active
```

```
> search Extended License is active  
searching all memory...
```

```
matches for: 457874656e646564204c6963656e736520697320616374697665 (b'Extended License is active')
```

```
0x00708120: -rwx loc_00708120 + 0x0
```

```
done (1 results)
```

```
6b10e3 WIFI_IP  
6b10eb BT_SEL  
6b10f3 J2534_SEL  
6b10ff CONNECTION_TYPE  
6b110f DEBUG_MODE  
6b111b DEMO_MODE  
6b1127 \T0AD\  
6b112f /T0AD/  
6b1137 totalcardiagnostics  
6b114b Usage Guide.url  
6b115b BuildLabEx  
6b1167 CurrentBuild  
6b1177 ReleaseId  
6b1183 CurrentVersion  
6b1193 InstallationType  
6b11a7 EditionID  
6b11b3 ProductName  
6b121f Windows info:  
6b122f Registry Key:  
6b123f Registry ID:  
6b124f KeyV2  
6b1257 --Restart  
6b1267 : %i.%i.%i  
6b1273 Extended License is active  
6b128f Checking for Extended License status...  
6b12b7 Expiry date: %04i-%02i-%02i  
6b12d3 Extended License key: %s
```

arted

VivisectION Find Strings (new last week)

Vivisect: temp_2_1.viv

File Edit View Share Tools Window Plugins

Ion tools: CLI Ion>

History ▾ 0x75d2b4

0x75d2a4 44 50 46 5f 53 4f 4f 54 5f 50 43 54
0x75d2b4 53 45 43 55 52 49 54 59 5f 41 52 52
0x75d2c4 53 57 44 4c 5f 53 45 43 5f 4c 45 56
0x75d2d4 53 57 44 4c 49 44 5f 51 56 5f 41 52 52 41 59 00 SWDLID_QV_ARRAY.
0x75d2e4 53 57 44 4c 49 44 5f 41 52 52 41 59 00 00 00 00 SWDLID_ARRAY....
0x75d2f4 53 57 44 4c 49 44 32 5f 51 56 00 00 53 57 44 4c SWDLID2_QV..SWDL
0x75d304 49 44 32 5f 51 54 00 00 53 57 44 4c 49 44 32 00 ID2 OT..SWDLID2.

PrintDiscoveredStats
Reset Console In Use
Scan for Strings
Scan for Pointers
Stack >

"Find Strings" settings

Find Strings Dialog

Min String Length

ALL Memory (ignore other fields)

Starting Map

Ending Map

Start Address

Stop Address

Apply Strings

String Type

Ion tools: CLI

viv: 0x75d2b4

History ▾

0x75d2b4

256

bytes ▾

Opts ▾

0x75d2a4	44 50 46 5f 53 4f 4f 54 5f 50 43 54 5f 43 4c 00	DPF_S00T_PCT
0x75d2b4	53 45 43 55 52 49 54 59 5f 41 52 52 41 59 00 00	SECURITY_ARR
0x75d2c4	53 57 44 4c 5f 53 45 43 5f 4c 45 56 45 4c 00 00	SWDL_SEC_LEV
0x75d2d4	53 57 44 4c 49 44 5f 51 56 5f 41 52 52 41 59 00	SWDLID_QV_AR
0x75d2e4	53 57 44 4c 49 44 5f 41 52 52 41 59 00 00 00 00	SWDLID_ARRAY
0x75d2f4	53 57 44 4c 49 44 32 5f 51 56 00 00 53 57 44 4c	SWDLID2_QV..
0x75d304	49 44 32 5f 51 54 00 00 53 57 44 4c 49 44 32 00	ID2_QT..SWDL
0x75d314	53 57 44 4c 49 44 00 00 50 52 45 53 50 45 52 49	SWDLID..PRES
0x75d324	4f 44 00 00 50 52 45 53 44 41 54 41 00 00 00 00	OD..PRESDATA
0x75d334	53 45 53 53 49 4f 4e 5f 44 41 54 41 00 00 00 00	SESSION_DATA
0x75d344	53 45 43 44 41 54 41 00 4f 50 54 49 4f 4e 00 00	SECDATA.OPTI
0x75d354	52 4f 55 54 49 4e 45 5f 49 44 00 00 4c 41 42 45	ROUTINE_ID..
0x75d364	4c 5f 49 44 00 00 00 00 4e 41 4d 45 5f 49 44 00	L_ID....NAME
0x75d374	3a 20 25 2e 30 66 20 00 20 25 2e 30 66 20 00 00	: %.0f . %.0
0x75d384	20 25 2e 30 66 2e 2e 2e 25 2e 30 66 20 00 00 00	%.0f...%.0f
0x75d394	29 20 00 00 e0 9e 59 00 70 a2 59 00 70 b1 59 00	)Y.p.Y.

Discovered Strings:

0x76e904: 'READABLECODE'
0x76e914: ': %04hX'
0x76e91c: ': %04hX, '
0x76e92c: '0300%04X'
0x76e938: 'Unable to run erase flash command,
error: %i'
0x76e968: ' %i/%i...'
0x76e974: 'erase'
0x76e984: 'data_format_identifier'
0x76e9a4: '0301%08X'
0x76e9b8: 'Start downloading SBL %s'
0x76e9d4: 'MAX_CAN_BLOCK_SIZE'
0x76e9e8: 'Start downloading VBF %s'
0x76eadc: 'IVD initialization Base Class'
0x76ee7c: '\\n\\nOutcode: %06X'
0x76f0a4: '02A1%02X'
0x76f0b0: '0120%02X'
0x76f0c0: 'F01000'
0x76f0c8: '005010'

FuncGraph0: None (0x401000) viv: 0x75d2b4

0x00401000 is not in a function!
Welcome to Vivisect (Qt Edition)!
Random Tip: Hold <shift> to move using the mc

Python Interactive

applying strings to the workspace

- don't want all "strings" – some are code, others pointers

Min String Length	5
ALL Memory (ignore other fields)	No
Starting Map	0x400000: C:\\Program Files\\Target\\Target.exe
Ending Map	0x80a000: C:\\Program Files\\Target\\Target.exe
Start Address	0x7422c0
Stop Address	0x791e18
Apply Strings	Yes
String Type	ASCII
✓ OK ⊗ Cancel	

soooo... what's interesting?

File Edit View Share Tools Window Plugins

Ion tools: CLI

vet: str_SECURITY_ARRAY_0075d2b4

History ▾ 0x0075d2b4 256 Viv ▾ Opts ▾

:0x0075d28c	00	
:0x0075d28d	f5	
:0x0075d28e	58	'X'
:0x0075d28f	00	
:0x0075d290	40	'@'
:0x0075d291	00	
:0x0075d292	00	
:0x0075d293	00	
:0x0075d294	str_DPF_SOOT_PCT_OL_0075d294: [0 XREFS]	
:0x0075d294	'DPF_SOOT_PCT_OL\x00'	
:0x0075d2a4	str_DPF_SOOT_PCT_CL_0075d2a4: [0 XREFS]	
:0x0075d2a4	'DPF_SOOT_PCT_CL\x00'	
:0x0075d2b4	str_SECURITY_ARRAY_0075d2b4: [0 XREFS]	
:0x0075d2b4	'SECURITY_ARRAY\x00'	
:0x0075d2c3	00	
:0x0075d2c4	str_SWDL_SEC_LEVEL_0075d2c4: [0 XREFS]	
:0x0075d2c4	'SWDL_SEC_LEVEL\x00'	
:0x0075d2d3	00	
:0x0075d2d4	str_SWDLID_QV_ARRAY_0075d2d4: [0 XREFS]	
:0x0075d2d4	'SWDLID_QV_ARRAY\x00'	
:0x0075d2e4	str_SWDLID_QV_ARRAY_0075d2e4: [0 XREFS]	
:0x0075d2e4	'SWDLID_QV_ARRAY\x00'	

vet: str_SECURITY_ARRAY_0075d2b4 FuncGraph0: target0+0x283589 (0x683589) viv: 0x0075bf14 foo: 0x0075d294 bar: 0x467f70

HOW DO I FIND CODE!?

- x86 is a Variable Size Instruction-set
- Off by one byte → dramatically change meaning
- how might we **accurately** identify code?
 - deobfuscated (interesting) map in particular!

finding RETs

- 12 stack areas (for each thread's stack)
- many memory maps, most are anonymous
- but...how do i know if a pointer is a RET??

[Stack]	[Stack]_3	0x03f7f000	4096
[Stack]	[Stack]_4	0x03f80000	4096
	_340	0x03f90000	4096
[Stack]	[Stack]_5	0x0409c000	16384
[Stack]	[Stack]_6	0x040a0000	4096
	_341	0x040b0000	4096
	_342	0x040c0000	286720
counters.dat	counters.dat_0	0x041c0000	4096
[Stack]	[Stack]_7	0x042cf000	4096
[Stack]	[Stack]_8	0x043cf000	4096
[Stack]	[Stack]_9	0x044cf000	4096
[Stack]	[Stack]_10	0x045cf000	4096
[Stack]	[Stack]_11	0x046cf000	4096
[Stack]	[Stack]_12	0x046d0000	4096
	_343	0x046e0000	4096

- finding RETs - how do i know if a pointer is a RET??

- go to the pointer
- back up 2, 3, 4, 5, 6, 7, 8 bytes and disassemble
- is the instruction a CALL?
- does it end *at* the location of this pointer?

0x03f7fda4	00000000	02cb4e6c	03f7fdc0	777fc4a2
0x03f7fdb4	000000e4	ffffffff	00000000	03f7ff84
0x03f7fdc4	757817c4	0037ec9b	03f7ff18	02cb4a30
0x03f7fdd4	005143a6	000003e8	b248d3ed	00000000
0x03f7fde4	00000000	02cb4a30	00000000	00000000

History ▾ 0x5143a6 256 Viv ▾ Opts ▾

:0x0051439c	8e	
:0x0051439d	3c	'<'
:0x0051439e	04	
:0x0051439f	00	
:0x005143a0	00	
:0x005143a1	e8	
:0x005143a2	2a	'*'
:0x005143a3	37	'7'
:0x005143a4	18	
:0x005143a5	00	
:0x005143a6	e9	
:0x005143a7	99	
:0x005143a8	fe	
:0x005143a9	ff	
:0x005143aa	ff	
:0x005143ab	8b	
:0x005143ac	10	

```
:0x005143a1 e82a371800 call sub_00697ad0 ;sub_00697ad0(1000)
:0x005143a6 e999feffff jmp loc_00514244
:0x005143ab loc_005143ab: [1 XREFS]
:0x005143ab 8b16 mov edx,dword [esi]
:0x005143ad 8b4214 mov eax,dword [edx + 20]
:0x005143b0 8bce mov ecx,esi
:0x005143b2 899e24040000 mov dword [esi + 1060],ebx
:0x005143b8 ffd0 call eax ;UnknownApi()
:0x005143ba 6a01 push 1
:0x005143bc ff1518656e00 call dword [loc_006e6518] ;sub_73ab9feb(1)
:0x005143c2 b801000000 mov eax,1
:0x005143c7 8b4df4 mov ecx,dword [ebp + local16]
:0x005143ca 64890d00000000 fs: mov dword [0x00000000],ecx
:0x005143d1 59 pop ecx
:0x005143d2 5f pop edi
:0x005143d3 5e pop esi
:0x005143d4 5b pop ebx
:0x005143d5 8be5 mov esp,ebp
:0x005143d7 5d pop ebp
:0x005143d8 c3 ret
```

GO! GO! GO!

- ummmm, woops?

```
609444191 May 20 20:11 target-fromvsnap-07.viv
```

```
582M May 20 20:11 target-fromvsnap-07.viv
```

oops, too much code (not much worth seeing in NTDLL and Kernel32 and User32

- left overnight to analyze
- needed 80GB swapspace (crashed two nights in a row)
- ended up with a 500MB workspace file
 - 10 minutes to load locally
 - 45 minutes to load over the network!
- also makes FuncRecon completely useless (VivisectION)

oops, too much code (not much worth seeing in NTDLL and Kernel32 and User32

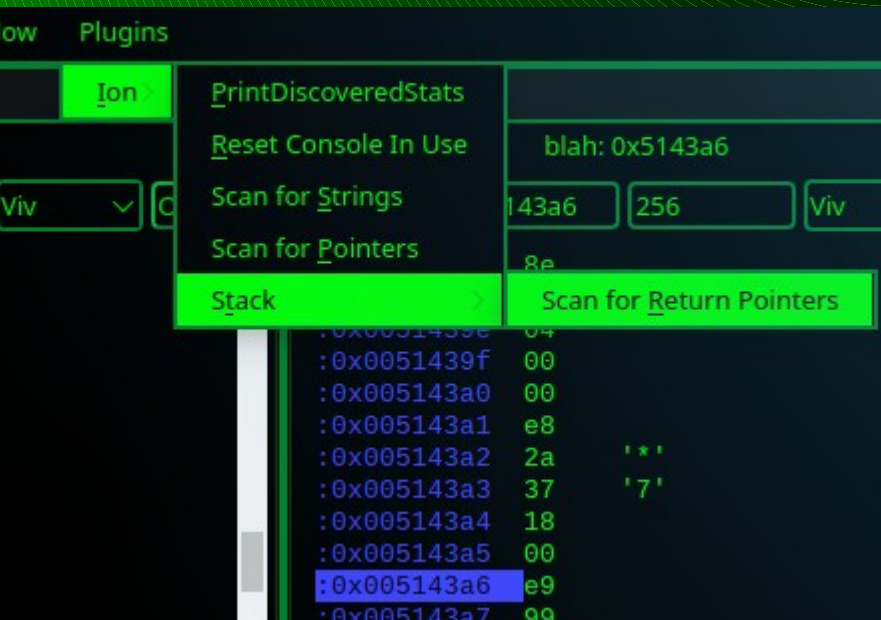
- the partial emulator emulates every path through every function:
 - NTDLL, User32, Kernel32, etc... **every** piece of code
 - it works **amazingly** well
 - by next week sometime
 - using all the RAMs as it stores interesting tidbits along the way

broke Vivisect Server

- would never load the workspace
- discovered a weakness in VivServer
 - too much time lapsed from connection
 - Server was queuing the ENTIRE 500MB before sending anything
- fixed...

kill all tedium and brain drain!

- added to VivisectIOn



ALL Memory (ignore other fields)	No
Starting Map	0x122000: [stack]
Ending Map	0x130000: [stack]
Start Address	0x0
Stop Address	0x0
Apply Pointers to Workspace	No
Starting Map (target)	0x400000: target0
Ending Map (target)	0x80a000: target0

☒ OK ☐ Cancel

next: lots of patterns that
"feel like pointers"

- huh, what have we here?
 - kinda looks like pointers?
- as i made them pointers...

```
:0x0075d280  a0
:0x0075d281  83
:0x0075d282  59  'Y'
:0x0075d283  00
:0x0075d284  50  'P'
:0x0075d285  86
:0x0075d286  59  'Y'
:0x0075d287  00
:0x0075d288  c0
:0x0075d289  dc
:0x0075d28a  58  'X'
:0x0075d28b  00
:0x0075d28c  d0
:0x0075d28d  f5
:0x0075d28e  58  'X'
:0x0075d28f  00
:0x0075d290  40  '@'
:0x0075d291  00
:0x0075d292  00
:0x0075d293  00
:0x0075d294  str_DPF_S00T_PCT_OL_0075d294: [0 >
:0x0075d294  'DPF_S00T_PCT_OL\x00'
:0x0075d2a4  str_DPF_S00T_PCT_CL_0075d2a4: [0 >
:0x0075d2a4  'DPF_S00T_PCT_CL\x00'
:0x0075d2b4  str_SECURITY_ARRAY_0075d2b4: [0 XF
:0x0075d2b4  'SECURITY_ARRAY\x00'
```

next: lots of patterns that
"feel like pointers"

- vast arrays of pointers to lots of code
- in the form of Vtables
 - an artifact of C++ classes/objects
- LOOK MA! I GOT XREFS!?

BAZ: str_SECURITY_ARRAY_0075d2b4		
x0075d2b4	256	
0x0075d24c	ptr: 0x0058e920	(sub_0058e920)
0x0075d250	ptr: 0x0058e890	(sub_0058e890)
0x0075d254	ptr: 0x0058d2e0	(sub_0058d2e0) ;void
0x0075d258	ptr: 0x0058d200	(sub_0058d200)
0x0075d25c	ptr: 0x0058e430	(sub_0058e430)
0x0075d260	ptr: 0x0060d600	(sub_0060d600)
0x0075d264	ptr: 0x00597b80	(sub_00597b80)
0x0075d268	ptr: 0x0058b9f0	(sub_0058b9f0)
0x0075d26c	ptr: 0x005996c0	(sub_005996c0)
0x0075d270	ptr: 0x00598210	(sub_00598210)
0x0075d274	ptr: 0x00599af0	(sub_00599af0)
0x0075d278	ptr: 0x00599cc0	(sub_00599cc0)
0x0075d27c	ptr: 0x0058c510	(sub_0058c510)
0x0075d280	ptr: 0x005983a0	(sub_005983a0)
0x0075d284	ptr: 0x00598650	(sub_00598650)
0x0075d288	ptr: 0x0058dcc0	(sub_0058dcc0)
0x0075d28c	ptr: 0x0058f5d0	(sub_0058f5d0)
0x0075d290	loc_0075d290: [1 XREFS]	
0x0075d290	40 '@' ;int	
0x0075d291	00	
0x0075d292	00	
0x0075d293	00	
0x0075d294	str_DPF_S00T_PCT_OL_0075d294: [1 XREFS]	
0x0075d294	'DPF_S00T_PCT_OL\x00'	
0x0075d2a4	str_DPF_S00T_PCT_CL_0075d2a4: [2 XREFS]	
0x0075d2a4	'DPF_S00T_PCT_CL\x00' ;int	
0x0075d2b4	str_SECURITY_ARRAY_0075d2b4: [2 XREFS]	
0x0075d2b4	'SECURITY_ARRAY\x00' ;int	

- new Viv Functionality: Make Pointer Array



Pointer Array Dialog

Start Address

Stop Address

Count (if nonzero, ignore stop address)

Stop if we run into a NULL pointer ☒ True

☒ OK ☐ Cancel

- Analysis of C++ objects...

- VTable Entries

- digging into specific functions
- discover how classes are laid out in memory
- Constructors and Destructors
- Automated Structure Reconstitution!

```
:0x0075d1c8 VTBL_____6__0075d1c8: [2 XREFS]
:0x0075d1c8 ptr: 0x005989f0 (DESTRUCTOR_005989f0)
:0x0075d1cc ptr: 0x00599100 (Init_StringMagicStuff_00599100)
:0x0075d1d0 ptr: 0x00596f60 (sub_00596f60)
:0x0075d1d4 ptr: 0x0058c040 (sub_0058c040)
:0x0075d1d8 ptr: 0x0040fc10 (sub_0040fc10)
:0x0075d1dc ptr: 0x00599550 (sub_00599550)
:0x0075d1e0 ptr: 0x0058ec60 (sub_0058ec60)
:0x0075d1e4 ptr: 0x00512ec0 (sub_00512ec0)
:0x0075d1e8 ptr: 0x0058eda0 (sub_0058eda0)
```

Analysis of C++ objects...

- referring to C++ Objects
 - what creates them?
 - trace VTables
 - follow the white rabbit
 - Class Inheritance!
 - destructors chaining tells a lot...

power of the DESTRUCTOR

- destructors have to clean up the object
- they are **very** telling about what is where, class inheritance, and embedded/included other classes

```
:0x005989f0  
:0x005989f0 FUNC: int thiscall_caller DESTRUCTOR_005989f0( void * ecx, int arg1, ) [1]  
:0x005989f0  
:0x005989f0 Stack Variables: (offset from initial top of stack)  
:0x005989f0 4: int arg1  
:0x005989f0  
:0x005989f0 56 push esi  
:0x005989f1 8bf1 mov esi,ecx  
:0x005989f3 e848dfffff call FREE_Obj__76d1c8__00596940 ;sub_00596940(ecx)  
:0x005989f8 f644240801 test byte [esp + arg1],1  
:0x005989fd 7409 jz loc_00598a08
```

```
:0x005989ff 56 push esi  
:0x00598a00 e8ac110e00 call Decr_Free_00679bb1 ;sub_00679bb1(ecx)  
:0x00598a05 83c404 add esp,4
```

```
:0x00598a08 loc_00598a08: [1 XREFS]  
:0x00598a08 8bc6 mov eax,esi  
:0x00598a0a 5e pop esi  
:0x00598a0b c20400 ret 4
```

:0x00596940 FUNC: int thiscall FREE_obj__76d1c8__00596940(void * ecx,) [11 XREFS]

:0x00596940

:0x00596940 Stack Variables: (offset from initial top of stack)

:0x00596940 -4: int local4

:0x00596940 -12: int local12

:0x00596940 -16: int local16

:0x00596940

:0x00596940 6aff push 0xffffffff

:0x00596942 68c7ec6c00 push loc_006cecc7

:0x00596947 64a100000000 fs: mov eax,dword [0x00000000]

:0x0059694d 50 push eax

:0x0059694e 51 push ecx

:0x0059694f 53 push ebx

:0x00596950 55 push ebp

:0x00596951 56 push esi

:0x00596952 57 push edi

:0x00596953 a1b0c27d00 mov eax,dword [loc_007dc2b0]

:0x00596958 33c4 xor eax,esp

:0x0059695a 50 push eax

:0x0059695b 8d442418 lea eax,dword [esp + local12]

:0x0059695f 64a300000000 fs: mov dword [0x00000000],eax

:0x00596965 8bf1 mov esi,ecx

:0x00596967 89742414 mov dword [esp + local16],esi

:0x0059696b c706c8d17500 mov dword [esi],VTBL_____6__0075d1c8

:0x00596971 bf10000000 mov edi,16

:0x00596976 39be60020000 cmp dword [esi + 608],edi

:0x0059697c c744242007000000 mov dword [esp + local4],7

:0x00596984 720f jc loc_00596995

```
:0x00596986 8b864c020000    mov eax,dword [esi + 588]
:0x0059698c 50              push eax
:0x0059698d e81f320e00      call Decr_Free_00679bb1 ;sub_00679bb1(0x61616161)
:0x00596992 83c404          add esp,4
```

```
:0x00596995 loc_00596995: [1 XREFS]
:0x00596995 33db           xor ebx,ebx
:0x00596997 bd0f000000     mov ebp,15
:0x0059699c 89ae60020000   mov dword [esi + 608],ebp
:0x005969a2 899e5c020000   mov dword [esi + 604],ebx
:0x005969a8 889e4c020000   mov byte [esi + 588],bl
:0x005969ae 39be44020000   cmp dword [esi + 580],edi
:0x005969b4 720f           jc loc_005969c5
```

EDI is 16

```
:0x005969b6 8b8630020000    mov eax,dword [esi + 560]
:0x005969bc 50              push eax
:0x005969bd e8ef310e00      call Decr_Free_00679bb1 ;sub_00679bb1(0x61616161)
:0x005969c2 83c404          add esp,4
```

```
:0x005969c5 loc_005969c5: [1 XREFS]
:0x005969c5 89ae44020000    mov dword [esi + 580],ebp
:0x005969ch 899e40020000    mov dword [esi + 576],ebx
```

```

loc_005969c5: [1 XREFS]
89ae44020000    mov dword [esi + 580],ebp
899e40020000    mov dword [esi + 576],ebx
8d8ee0010000    lea ecx,dword [esi + 480]
889e30020000    mov byte [esi + 560],bl
e87e46ffff     call sub_0058b060 ;sub_0058b060(ecx)
39bedc010000    cmp dword [esi + 476],edi
720f           jc loc_005969f9

```

```

c8010000    mov eax,dword [esi + 456]
push eax
310e00     call Decr_Free_00679bb1 ;sub_00679bb1(0x61616161)
04         add esp,4

```

```

005969f9 loc_005969f9: [1 XREFS]
005969f9 89aedc010000    mov dword [esi + 476],ebp
005969ff 899ed8010000    mov dword [esi + 472],ebx
00596a05 8dbeb4010000    lea edi,dword [esi + 436]
00596a0b 889ec8010000    mov byte [esi + 456],bl
00596a11 8b4704         mov eax,dword [edi + 4]
00596a14 3bc3          cmp eax,ebx
00596a16 741c          jz loc_00596a34

```

- Embedded Object

- Another STRING

- Something different
- (EBX == 0)

```
18 8b542414      mov edx,dword [esp + local16]
1c 8b4f08        mov ecx,dword [edi + 8]
1f 52           push edx
20 57           push edi
21 51           push ecx
22 50           push eax
23 e818dbe6ff    call FREE_CppString_Array_00404540 ;sub_00404540(0x61616161,0x6
28 8b4704        mov eax,dword [edi + 4]
2b 50           push eax
2c e880310e00    call Decr_Free_00679bb1 ;sub_00679bb1(0x61616161)
31 83c414        add esp,20
```

```
:0x00596a34 loc_00596a34: [1 XREFS]
:0x00596a34 895f04        mov dword [edi + 4],ebx
:0x00596a37 895f08        mov dword [edi + 8],ebx
:0x00596a3a 895f0c        mov dword [edi + 12],ebx
:0x00596a3d 8b86a8010000  mov eax,dword [esi + 424]
:0x00596a43 3bc3         cmp eax,ebx
:0x00596a45 7409         jz loc_00596a50
```

```
:0x00596a47 50           push eax
:0x00596a48 e864310e00    call Decr_Free_00679bb1 ;sub_00679bb1(0x61616161)
:0x00596a4d 83c404        add esp,4
```

```
:0x00596a50 loc_00596a50: [1 XREFS]
```

recap C++ structure reconstitution

- look for bunches of **function pointers**
- **xrefs to the start** of a bunch
 - beginning of VTABLE?
- analyze these functions for **offsets from THIS** argument
 - rebuilding the data structure
- analyze the first function in the VTABLE
 - often this is the **Destructor**
 - process of “destructing” tells many tales
 - like where embedded classes are
 - like class inheritance and hierarchy
 - and a connection of all the vtables
 - where everything comes from
 - great use of **Graphs**...(ie. Graph Theory)

conclusion:

- what, you wanted to see the SEED/Key Exchange?
- lol, i'm a **mentor**, not a dumper of free 0-day
- you are better served by going and finding it **yourself!**
- so go! i taught you to fish!

real conclusion

- **think** outside the box
 - make your **tools** do **new** things
 - understand **why** things work the way they do
 - all of that leads to a **powerful capability**...
-
- then go **hunting** for what you want
 - **imagine** what it **might** look like
 - **you** are the real power
 - go forth and do great things...

